63. WSL 2: Real Linux Inside Windows

Builds on **#50 (Docker)**, **#53 (Kubernetes)** and **#59 (local setup)**.

> **In one line: WSL 2** (Windows Subsystem for Linux, version 2) lets you run a **real Linux (e.g. Ubuntu)** inside Windows, at the **same time**, with **no dual boot** and almost **no setup**. Servers run Linux, so you develop the way production works.

1. Real-life example: a flat inside your house

> Your house is **Windows**. You want a **Linux** lifestyle too.
> **Dual boot** = two separate houses. You must **leave one** (restart) to live in the other.
> **Normal VM (VirtualBox)** = a heavy caravan parked in your garden. Slow to start, and it takes fixed space.
> **WSL 2** = a **small flat built inside your house**, with its own door. It opens in **2 seconds**, shares the electricity (RAM/CPU) only when used, and you can walk between rooms (files) freely.

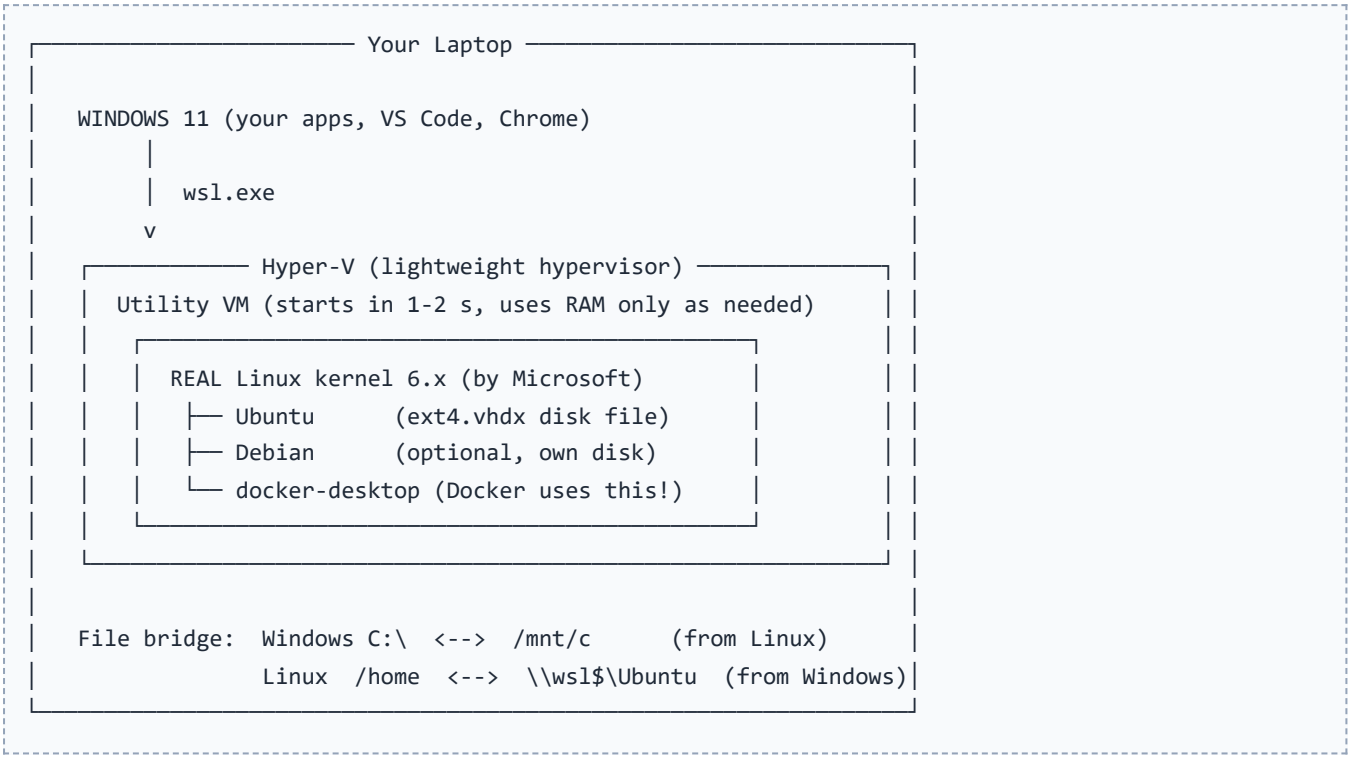
2. Why WSL was made (history)

Year	What happened
Before 2016	Windows developers used Cygwin, Git Bash, VirtualBox or dual boot. All slow or clumsy.
2016	**WSL 1** launched: Windows **translates** Linux commands into Windows ones. No real Linux kernel.
2019	**WSL 2** announced: a **real Linux kernel** in a tiny, fast VM. Much faster and fully compatible.
2020-21	Docker Desktop switches to WSL 2. `wsl --install` becomes one command. GUI Linux apps (WSLg) arrive.
2022+	WSL moves to the **Microsoft Store / its own package**, updated separately from Windows. Made open source in 2025.

Why developers need it: about 90% of servers, Docker images and cloud machines run **Linux**. Tools like bash, apt, Redis, Kubernetes tools, Python ML libraries and shell scripts work best (or only) on Linux.

PART 1: How it works

3. Inside the machine



Piece	Meaning
Hyper-V / Virtual Machine Platform	The Windows feature that runs tiny virtual machines
Utility VM	One hidden, lightweight VM shared by all your Linux distros
Linux kernel	The real "engine" of Linux, shipped and updated by Microsoft
Distro	The Linux "flavour" (Ubuntu, Debian, Alpine...): its tools and package manager
ext4.vhdx	A single file on C: that is the distro's whole hard disk. It grows as you add files.
WSLg	Lets Linux GUI apps open as normal Windows windows

4. WSL 1 vs WSL 2

	WSL 1	WSL 2
How	Translates Linux system calls into Windows	A **real Linux kernel** in a light VM
Compatibility	Many tools break (Docker doesn't work)	**Almost 100%**. Docker, systemd and K8s tools work.
Speed in Linux files	Slow	**Fast** (2-5x faster for npm install, git)
Speed on Windows files (`/mnt/c`)	Faster	Slower (files cross a bridge)
Use today?	Rarely	**Yes, the default**

5. WSL 2 vs other options

	WSL 2	VirtualBox / VMware	Dual boot	Docker
Startup	About 2 s	About 30-60 s	Restart PC	About 1 s
RAM	Takes and returns as needed	Fixed block	Full PC	Small
Use Windows at the same time	Yes	Yes	No	Yes
Full Linux desktop	Apps only (WSLg)	Yes	Yes	No
Purpose	**Daily dev environment**	Testing other OSes	Gaming / full Linux	**Packaging one app**

> **WSL 2 vs Docker:** they are **not rivals**. WSL 2 is your **Linux workspace** (like a full kitchen). Docker runs **packed apps** (like tiffin boxes). On Windows, **Docker Desktop itself runs on WSL 2** (the `docker-desktop` distro).

PART 2: Install (what we did on this laptop)

6. Requirements

- Windows 10 (2004+) or Windows 11. This laptop: **Windows 11, build 26300**
- **Virtualization enabled** in BIOS (Intel VT-x / AMD-V)
- Windows features: **Windows Subsystem for Linux** + **Virtual Machine Platform**
- Free disk space (Ubuntu starts at about 1.3 GB). This laptop: 142 GB free on C:

7. The normal way: one command

```
# PowerShell as Administrator
wsl --install                # enables features + installs WSL + Ubuntu
# restart the PC if it asks, then:
wsl                          # first launch: create a Linux username + password
```

8. What actually happened here (real troubleshooting)

```
1. wsl --status          -> "The system cannot find the file specified"
       |                    (wsl.exe was there, but the WSL program files were missing)
2. dism get-featureinfo -> WSL feature: Enabled, VirtualMachinePlatform: Enabled
       |                    so the Windows features were FINE
3. Get-AppxPackage       -> old WSL package 2.6.1 registered but BROKEN
       |
4. winget install Microsoft.WSL   -> failed: "administrator privileges are required"
   (run again as Admin)             -> Successfully installed WSL 2.7.14
       |
5. wsl --install -d Ubuntu        -> "A distribution with the supplied name already exists"
       |
6. wsl -d Ubuntu         -> "Failed to attach disk ...\ext4.vhdx: path not found"
       |                    (old Ubuntu entry, but its disk file was gone, so no data)
7. wsl --unregister Ubuntu  +  wsl --install -d Ubuntu  ->  fresh Ubuntu 26.04 LTS
       |
8. Running: WSL 2, kernel 6.18, 3.7 GB RAM available, 1 TB virtual disk
```

```
# the fix commands, in order
winget install --id Microsoft.WSL -e      # (in an Admin terminal) install/repair WSL itself
wsl --version                             # check: WSL 2.7.14, kernel 6.18...
wsl --set-default-version 2
wsl --unregister Ubuntu                   # ONLY because its disk was already missing
wsl --install -d Ubuntu
wsl -l -v                                 # Ubuntu   Stopped   2
```

> **Careful:** `wsl --unregister <name>` **deletes that Linux and all files inside it**. We only used it because the disk was already gone. Normally, back up first with `wsl --export Ubuntu backup.tar`.

9. First steps inside Ubuntu

```
wsl -d Ubuntu                # first time: create username (small letters) + password
                             # (the password is invisible while typing: that's normal)

sudo apt update && sudo apt upgrade -y          # update all packages
sudo apt install -y build-essential git curl     # basic dev tools
```

PART 3: Daily use

10. Must-know commands

Command (from Windows)	What it does
`wsl`	Open the default Linux
`wsl -d Ubuntu`	Open a specific distro
`wsl -l -v`	List distros, state (Running/Stopped) and version
`wsl -l -o`	List distros you can install
`wsl --shutdown`	Stop everything and give back the RAM
`wsl --terminate Ubuntu`	Stop one distro
`wsl --update`	Update WSL and the kernel
`wsl --export Ubuntu D:\ubuntu.tar`	Back up the whole distro
`wsl --import Ubuntu2 D:\wsl\u2 D:\ubuntu.tar`	Restore it (also a way to move it to D:)
`wsl ls -la`	Run one Linux command from Windows

11. Files: two worlds, one bridge

```
From LINUX you see Windows drives:        From WINDOWS you see Linux files:
  /mnt/c/Users/gouri                        \\wsl$\Ubuntu\home\gourish
  /mnt/d/nest-js/my-app                     (paste into Explorer, or run
                                             "explorer.exe ." inside Linux)
```

> **Speed rule:** keep projects **inside Linux** (`~/projects/my-app`) when you work with Linux tools. `npm install` and `git status` on `/mnt/d/...` cross the bridge and can be **3-10x slower**. Use `/mnt/d` only for occasional access.

12. Node + NestJS in WSL

```
# install Node with nvm (the Linux-friendly way)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.bashrc
nvm install --lts
node -v

# put the project in the Linux file system (fast)
mkdir -p ~/projects && cd ~/projects
git clone https://github.com/gourishnaik/my-app.git
cd my-app && npm install && npm run start:dev

# open it in VS Code (installs the WSL extension automatically)
code .
```

Then open **http://localhost:3000** in Windows Chrome. WSL forwards `localhost` automatically.

13. VS Code + WSL

```
VS Code window (Windows)  ──────>  VS Code Server (inside Ubuntu)
UI, typing, extensions UI           runs: terminal, node, git, ESLint, debugger
                                    on Linux files at full speed

Bottom-left shows:  "WSL: Ubuntu"
```

14. Docker Desktop + WSL 2

Docker Desktop → **Settings** → **General**: "Use the WSL 2 based engine". Then **Settings** → **Resources** → **WSL Integration**: turn on **Ubuntu**.

```
# now inside Ubuntu:
docker ps
```

```
docker run -d -p 6379:6379 redis       # Redis for #49 / #54
kubectl get nodes                      # if Kubernetes is enabled (#53, #59)
```

15. Limit RAM and CPU: `.wslconfig`

WSL 2 can use up to **half your RAM** by default. On a laptop with little free RAM, set a limit. Create `C:\Users\gouri\.wslconfig`:

```
[wsl2]
# max RAM for all Linux distros (+ Docker)
memory=3GB
# max CPU cores
processors=4
swap=2GB

[experimental]
# give unused RAM back to Windows
autoMemoryReclaim=gradual
```

Put comments on their **own line**. Text after a value on the same line can be read as part of the value.

```
wsl --shutdown      # restart WSL so the settings apply
```

PART 4: Problems and fixes

16. Common errors

Error	Cause	Fix
"The system cannot find the file specified"	WSL package missing / broken (our case)	`winget install Microsoft.WSL` as Admin, or `wsl --update`
"administrator privileges are required"	Installer needs Admin	Run the terminal as Administrator
"Failed to attach disk ... ext4.vhdx"	The distro's disk file was moved/deleted (our case)	Restore the .vhdx, or `--unregister` + reinstall
"A distribution with the supplied name already exists"	An old entry has the same name	Use it, unregister it, or install with `--name`
0x80370102 "virtual machine could not be started"	Virtualization off in BIOS	Enable Intel VT-x / AMD-V in BIOS
Laptop slow, high RAM (`vmmem`)	WSL / Docker holding memory	`wsl --shutdown`, set `.wslconfig` memory
npm install very slow	Project on `/mnt/c` or `/mnt/d`	Move the project into `~/`
C: drive filling up	The ext4.vhdx grows but never shrinks by itself	Clean up inside Linux, then compact the vhdx, or move the distro to D: with export/import

17. Summary

- **WSL 2** = a real Linux kernel in a tiny, fast VM inside Windows. No dual boot, and it starts in seconds.
- **WSL 1** translated commands, while **WSL 2** runs real Linux, so it's faster and Docker works. Always use WSL 2.
- Install: `wsl --install`. If broken: `winget install Microsoft.WSL` (Admin), then `wsl --install -d Ubuntu`.
- Each distro lives in one **ext4.vhdx** file. `--unregister` deletes it, and `--export` backs it up.
- Files: `/mnt/d/...` from Linux, `\\wsl$\Ubuntu` from Windows. Keep projects **inside Linux** for speed.
- Dev setup: nvm + Node, VS Code `code .`, Docker Desktop WSL integration, `localhost` works from Windows.
- Control resources with `.wslconfig`, and free RAM with `wsl --shutdown`.